22 — SQLite Grundlagen

Die Datenbank aus Python steuern: CREATE, INSERT, SELECT, UPDATE, DELETE

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Warum eine Datenbank statt einer Datei?
2. Das Rezept: connect → cursor → execute → commit → close
3. `CREATE TABLE` – Struktur festlegen
4. `INSERT` – Daten sicher einfügen (?-Platzhalter)
5. `SELECT` – Daten abfragen (fetchone/fetchall)
6. `UPDATE` / `DELETE` – ändern und löschen
7. **Klassiker: Produkt-DB CRUD** – live programmiert

Lernziel: Eine SQLite-Datenbank aus Python anlegen, befüllen, abfragen und ändern – und **verstehen**, was DBMS, Puffer und Cursor dabei wirklich tun.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 22).

Warum eine Datenbank statt einer Datei?

Ohne DB (Foliensatz 12): selbst suchen

```
with open("produkte.txt") as f:
    for zeile in f:
        name, preis = zeile.split(";")
        if float(preis) < 100:
            print(name)
```

Mit DB: das DBMS suchen lassen

```
cur.execute("""SELECT name FROM produkt
               WHERE preis < 100""")
```

FS 20 (wozu ein DBMS) und FS 21 (Tabellen auf Papier) werden heute **ausführbar**: echte Tabellen, gesteuert aus Python.

SQLite – die ganze Datenbank in einer Datei

```
import sqlite3
conn = sqlite3.connect("veggie_soles.db")
```

- **SQLite** ist ein komplettes DBMS – als Modul in Python **eingebaut**: kein Server, keine Installation
- `connect()` öffnet die Datenbank**datei** – und legt sie an, wenn es sie noch nicht gibt
- Steckt in Firefox, Android, iOS – ideal für kleine Anwendungen, Prototypen, Lehre
- Spezialfall `":memory:"`: Datenbank nur im RAM – zum Ausprobieren, weg nach Programmende

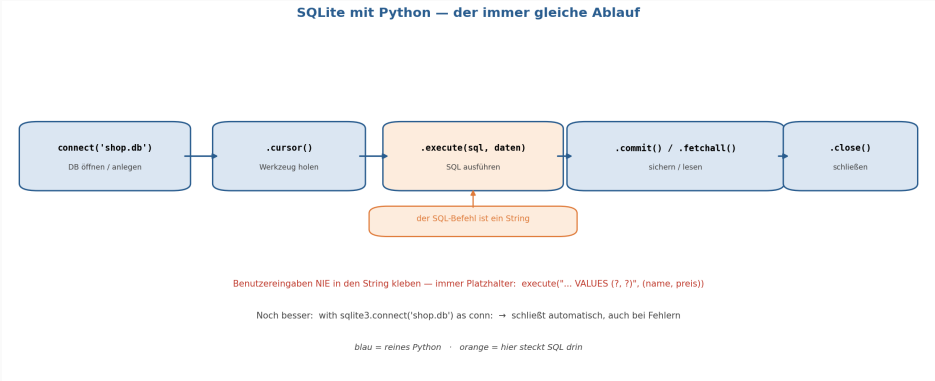
Unter der Haube: eine Datei – aber ein DBMS davor

`veggie_soles.db` liegt neben dem Skript, wie `log.txt` in FS 12. Der Unterschied: **wer** die Datei anfasst.

	`open(...)` (FS 12)	`sqlite3` (heute)
Sie bekommen	rohe Zeichen	fertige Zeilen (Tupel)
Suchen/Filtern	selbst programmieren	`WHERE` – macht das DBMS
Dateiformat	Ihr Problem	Sache des DBMS

Das Schichtenbild aus FS 20 wird konkret: **Ihr Skript** → **DBMS (`sqlite3`)** → **Datei**. Sie sprechen nie direkt mit der Datei – nur mit dem "Bibliothekar".

Das Rezept – der immer gleiche Ablauf



▶ **Live-Demo** – `01_verbindung_und_tabelle.py`

✎ **Sofort ausprobieren** – Notebook 22, Übung 1.1: Verbindung zu `uebung.db`, Cursor holen, schließen

CREATE TABLE – Struktur festlegen

```
cur.execute("""CREATE TABLE IF NOT EXISTS produkt (
    produkt_id INTEGER PRIMARY KEY,
    name       TEXT NOT NULL,
    preis      REAL)""")
conn.commit()
```

- Der SQL-Befehl ist ein **String** für `execute()`
- Pro Spalte: Name + Typ – SQLite kennt nur **fünf**: `INTEGER`, `REAL`, `TEXT`, `BLOB`, `NULL`
- `INTEGER PRIMARY KEY`: der Primärschlüssel aus FS 21 – zählt automatisch hoch
- `NOT NULL` = Pflichtfeld · `IF NOT EXISTS` = kein Fehler beim zweiten Lauf

Unter der Haube: was bei `execute()` passiert

```
cur.execute("SELECT name FROM produkt WHERE ...")
```

Schritt im DBMS	was passiert
1. Parser	SQL-String lesen, Syntax prüfen
2. Planer	Ausführungsplan wählen: *wie* suchen?
3. Ausführung	Zeilen aus der Datei holen/ändern
4. Ergebnis	Treffer im Cursor bereitstellen

SQL ist **deklarativ** (Paradigmen-Landkarte, FS 01): Sie sagen *was* Sie wollen – das DBMS entscheidet *wie*. Bei `open()` war der Suchplan Ihr Code; hier schreibt ihn das DBMS.

INSERT – Daten einfügen

```
sql = "INSERT INTO produkt(name, preis) VALUES(?,?)"
cur.execute(sql, ("Eco-Sneaker", 89.95))
cur.executemany(sql, [("Hemp-High", 109.00),
                      ("Bambus-Boot", 135.50)])
conn.commit()
```

- Werte als **Tupel** übergeben – für jedes ? genau einer, in Reihenfolge
- `executemany()`: eine Liste von Tupeln, ein Befehl – perfekt für Stammdaten
- `cur.lastrowid`: die zuletzt vergebene ID · erst `commit()` macht alles **dauerhaft**

Unter der Haube: `commit()` und der Puffer

```
cur.execute(sql, ("Eco-Sneaker", 89.95))
# Programm bricht HIER ab? -> Zeile ist weg
conn.commit()
# ab jetzt: dauerhaft in der .db-Datei
```

Zeitpunkt	steht es in der Datei?
nach `execute(INSERT ...)`	nein – nur in der offenen Transaktion
Absturz vor `commit()`	Änderung verworfen
nach `commit()`	ja – dauerhaft

Dasselbe Muster wie in FS 12 (`write()` → Puffer, erst `close()` sichert) – und genau das **D**urability aus ACID (FS 21): erst die Bestätigung macht die Änderung dauerhaft.

Unter der Haube: warum ? statt String-Bau

```
name = input("Suche: ")
sql = f"... WHERE name = '{name}'"    # NIE so!
# Eingabe:  x'; DELETE FROM produkt; --
# -> aus 1 Befehl werden 2: SELECT + DELETE!
```

	String-Bau (f-string/+)	Platzhalter ?
Eingabe wird	Teil des **Befehls**	immer nur ein **Wert**
böse Eingabe	wird ausgeführt	harmloser Suchtext

SQL-Injection – einer der häufigsten Angriffe im Web. Das DBMS setzt ?-Parameter erst **nach** dem Parsen ein, sie können also nie zu Befehlen werden. Deshalb: Werte **immer** als ? übergeben.

INSERT – Demo & Übung

▶ **Live-Demo** – `02_daten_einfuegen.py`

✎ **Sofort ausprobieren** – Notebook 22, Übungen 2.1 & 3.1: Tabelle `studenten` anlegen, drei Studierende **sicher** einfügen

SELECT – Daten abfragen

```
cur.execute("""SELECT name, preis FROM produkt
               WHERE preis < ?""", (100.0,))
for name, preis in cur.fetchall():
    print(name, preis)
```

Aufruf	liefert
`fetchone()`	die nächste Zeile (oder `None`)
`fetchall()`	alle (restlichen) Zeilen als Liste
`fetchmany(n)`	die nächsten n Zeilen

▶ **Live-Demo** – `03_abfragen_select.py`

Unter der Haube: der Cursor ist ein Ergebnis-Zeiger

```
cur.execute("SELECT name FROM produkt")  # 3 Treffer
```

Aufruf	liefert	Zeiger danach
`fetchone()`	`("Eco-Sneaker",)`	vor Zeile 2
`fetchone()`	`("Hemp-High",)`	vor Zeile 3
`fetchall()`	`[("Bambus-Boot",)]`	am Ende
`fetchall()`	`[]` – leer!	am Ende

- Wie der **Datei-Cursor** aus FS 12: jedes `fetch` bewegt den Zeiger weiter – nochmal lesen heißt: Abfrage neu `execute()`n

✎ **Sofort ausprobieren** – Notebook 22, Übung 4.1: alle Studierenden aus “Informatik” abfragen

UPDATE – Werte ändern

Sommerrabatt bei Veggie Soles: Eco-Sneaker 89.95 → 84.95 EUR

```
sql = """UPDATE produkt SET preis = ?
         WHERE name = ?"""
cur.execute(sql, (84.95, "Eco-Sneaker"))
conn.commit()
print(cur.rowcount, "Zeile(n) geaendert")
```

- SET: **welche Spalte(n)** neu – `WHERE`: **welche Zeilen** betroffen
- `cur.rowcount`: wie viele Zeilen wirklich getroffen wurden (Kontrolle!)
- Auch hier: ohne `commit()` nicht dauerhaft

Unter der Haube: `WHERE` entscheidet, wen es trifft

Das DBMS prüft die Bedingung **für jede Zeile** – wie ein `if` im `for` (FS 09/10), nur deklarativ:

Zeile	`name = 'Eco-Sneaker'`?	`UPDATE` wirkt?
1 Eco-Sneaker	wahr	`preis` → 84.95
2 Hemp-High	falsch	unverändert
3 Bambus-Boot	falsch	unverändert

Ohne `WHERE` ist jede Zeile ein Treffer: `UPDATE produkt SET preis = 0` ruiniert alle Preise, `DELETE FROM produkt` leert die Tabelle. Profi-Trick: das `WHERE` erst als `SELECT` testen, dann ändern.

DELETE – Zeilen löschen

```
cur.execute("DELETE FROM produkt WHERE name = ?",
            ("Bambus-Boot",))
conn.commit()
cur.execute("SELECT name FROM produkt")
print(cur.fetchall())        # Gegenprobe!
```

▶ **Live-Demo** – `05_update_delete.py`

✎ **Sofort ausprobieren** – Notebook 22, eigene Zelle: in Ihrer `studenten`-Tabelle per `UPDATE` ein Semester ändern, per `DELETE` eine Person entfernen – `SELECT` als Gegenprobe

Saubere Patterns: `with` und `try/except`

```
with sqlite3.connect("shop.db") as conn:
    conn.execute(sql, daten)  # commit automatisch
try:
    cur.execute(sql, daten)
except sqlite3.IntegrityError:
    print("Regel verletzt (UNIQUE/NOT NULL)")
```

- `with` wie in FS 12: am Blockende automatisch `commit()` – bei Exception `rollback()` (das **A** aus ACID: ganz oder gar nicht)
- `try/except` aus FS 11 fängt verletzte Tabellen-Regeln sauber ab

▶ **Live-Demo** – `04_with_und_fehler.py`

Klassiker: Produkt-DB CRUD – die Aufgabe

- Tabelle `produkte` (`produkt_id`, `name`, `preis`) in `produkte.db`
- die drei Stammprodukte einfügen (?-Platzhalter!), alle ausgeben
- Sommerrabatt: Eco-Sneaker 89.95 → 84.95 (`UPDATE`)
- Bambus-Boot auslisten (`DELETE`) + Gegenprobe

So soll es laufen:

```
Alle Produkte:
  1 | Eco-Sneaker  |  89.95 EUR
  2 | Hemp-High    | 109.00 EUR
  3 | Bambus-Boot  | 135.50 EUR
Sommerrabatt: Eco-Sneaker jetzt 84.95 EUR
Bambus-Boot ausgelistet. Gegenprobe:
  1 | Eco-Sneaker  |  84.95 EUR
  2 | Hemp-High    | 109.00 EUR
```

Klassiker: Produkt-DB CRUD – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_produkt_crud.py`
Teilziele:

1. `connect("produkte.db")` + Aufräumstart: `DROP TABLE IF EXISTS` → `CREATE TABLE`
2. Die drei Stammprodukte mit `executemany` und `?` einfügen, `commit()`
3. `SELECT` alle + formatiert ausgeben (f-Strings)
4. `UPDATE`: Eco-Sneaker auf 84.95, `commit()`, `rowcount` prüfen
5. `DELETE` Bambus-Boot + `SELECT`-Gegenprobe, `close()`

✎ **Zum Selberlösen** – Handout *“Klassiker 22: Adressbuch”* (`Klassiker/22_Klassiker_Adressbuch.pdf`): dieselbe Choreografie mit Kontakten statt Produkten. Vertiefung: Notebook 22, Abschlussübungen.

Cheat Card

Aufgabe	Code
Verbindung + Cursor	`sqlite3.connect("shop.db")` · `.cursor()`
Tabelle anlegen	`CREATE TABLE IF NOT EXISTS ...`
Einfügen (sicher)	`INSERT ... VALUES(?,?)` + Tupel
Abfragen	`SELECT ... WHERE x = ?`, dann `fetchall()`
Ändern	`UPDATE t SET s = ? WHERE ...`
Löschen	`DELETE FROM t WHERE ...`
Sichern / Schließen	`conn.commit()` · `conn.close()` / `with`
Klassiker heute	Produkt-DB: der komplette CRUD-Zyklus

Faustregel: Werte immer als `?`. Nach jeder Änderung `commit()`. Vor `UPDATE`/`DELETE` das `WHERE` doppelt lesen.

Ausblick: Notebook 23 – SQLite Vertiefung

Unsere Produkt-DB kann jetzt CRUD – aber *“Welche Kundin hat am meisten bestellt?”* braucht **zwei Tabellen gleichzeitig** und **Summen**:

```
SELECT k.name, SUM(b.betrag) AS umsatz
FROM   kunde k JOIN bestellung b
       ON k.kunde_id = b.kunde_id
GROUP BY k.kunde_id ORDER BY umsatz DESC
```

- `JOIN`: Tabellen über Fremdschlüssel verbinden – die Beziehungen aus FS 21/21b werden abfragbar
- `GROUP BY` + `SUM`/`AVG`/`COUNT`: Auswertungen · `ORDER BY`, `LIKE`, `BETWEEN`

Nur Vorschau – diese Syntax lernen Sie erst in Notebook 23. Heute genügt: es geht, und zwar deklarativ.

Heute geübt

✓ SQLite: das DBMS als Datei + Python-Modul – Schichtenbild aus FS 20 konkret
✓ Das Rezept: connect → cursor → execute → commit → close
✓ `execute()`-Pipeline: Parser → Plan → Zeilen – SQL ist **deklarativ**
✓ `commit()` und der Puffer – Durability zum Anfassen
✓ `?`-Platzhalter statt String-Bau – SQL-Injection verstanden
✓ Cursor = Ergebnis-Zeiger · `WHERE` entscheidet bei `UPDATE`/`DELETE`
✓ **Klassiker Produkt-DB CRUD** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Adressbuch** eigenständig lösen (Handout)
- “Sofort ausprobieren”-Aufgaben im Notebook 22 (Kap. 1–5)
- Abschlussübungen: eigene Datenbank `firma.db`, Notenverwaltung